AEP FROM ZERO • PART 1 • KNOW YOUR CUSTOMER (AEP)

EPISODE 14

AEP API Tutorial With Postman: Debug and RCA

How do you call Adobe Experience Platform APIs with Postman, debug a data incident, and write a root cause analysis? In this AEP API tutorial, Bloom proves its foundation through the API: the four headers on every Platform API call (Authorization Bearer token, x-api-key, x-gw-ims-org-id and x-sandbox-name), and the Profile Access API reading one customer by email and by a custom identity namespace. You will apply symptom, layer, evidence, fix to three real-shaped AEP troubleshooting

Previously: governed, then three complaints

- Last time: phone labelled C2, policy switched on
- Re-activated: hashed email and loyalty band only
- Today: three complaints from the CTO
- Proof through the API, not screenshots
- Method: symptom, layer, evidence, fix

KEY IDEA

Governance held last time. Today three complaints arrive a week before the relaunch, and the team must prove the foundation through the API.

Why prove it through the API?

Interface	API
What Adobe decided to show you	The object as it actually is
Screenshots someone must trust	A request anyone can run again
Great for working	Great for proof

KEY IDEA

The interface shows what Adobe decided to show you. The API shows the object as it actually is, which is why the CTO asked for it.

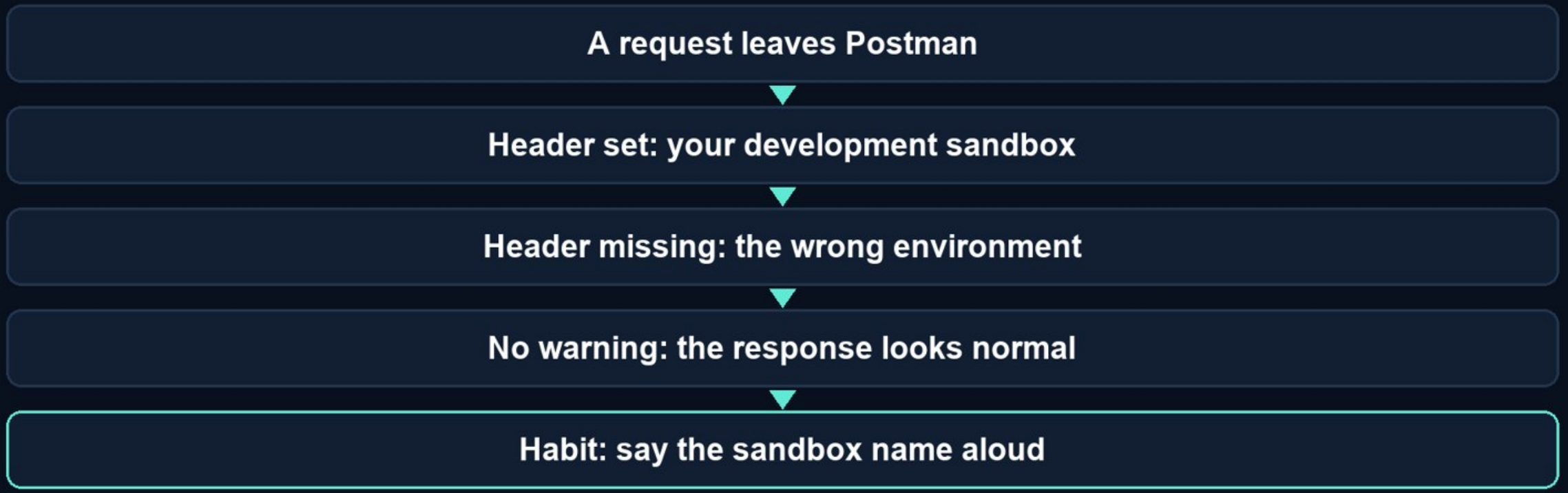
Four pieces of context on every call

Question	Header	Collection placeholder
Who are you?	Authorization: Bearer	{{ACCESS_TOKEN}}
Which application?	x-api-key	{{API_KEY}}
Which organisation?	x-gw-ims-org-id	{{IMS_ORG}}
Which sandbox?	x-sandbox-name	{{SANDBOX_NAME}}

KEY IDEA

Every Platform API call says who you are, which application is asking, which organisation, and which sandbox.

The sandbox header: forget it, hit production



KEY IDEA

The x-sandbox-name header decides which sandbox a call talks to. Forget it and you may query production while believing you are in development.

Profile Access API: one person, one lookup

- Request: GET /access/entities
- entityId = priya.sharma@example.com
- entityIdNS = email
- Plus the four headers on every call
- Returns the merged profile, as it is right now

KEY IDEA

To read one person, ask the Profile Access API for an entity by identity: a value plus the namespace that value belongs to.

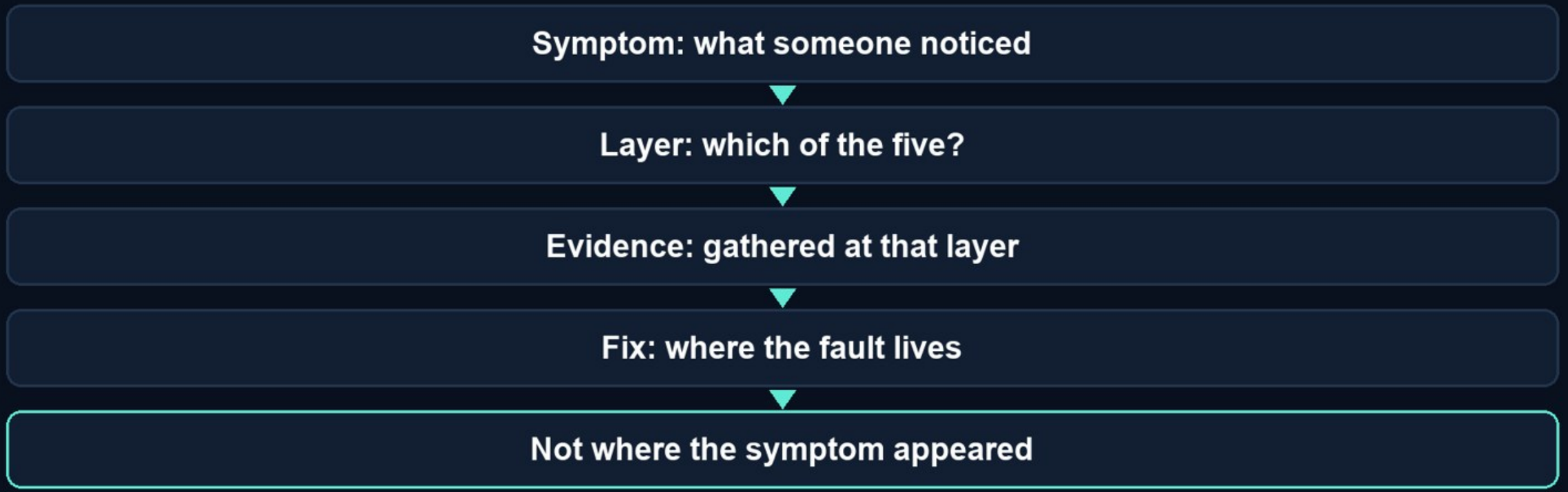
Same person, two namespaces

Lookup	Result
Email: priya.sharma@example.com	Priya Sharma, Gold, 2,340 points
BloomID: BLM-CUST-004217	The same profile
Identities on the profile	3: Email, BloomID, ECID

KEY IDEA

Fetched by email and then by BloomID, Priya came back as the same person with three identities. That proves the identity design.

Symptom, layer, evidence, fix



KEY IDEA

Complaints arrive as symptoms. Find the layer where the fault lives, gather evidence there, and fix it there, not where the symptom showed up.

Landmine one: the vanishing loyalty band

Step	Landmine one
Symptom	Loyalist file thinner every day
Evidence	Last week's Gold profile: band Gold
Evidence	Yesterday's: tier Gold, band null
Evidence	Mapping edited three days ago
Layer	Ingestion: the calculated field

KEY IDEA

Older profiles carry a loyalty band; newer ones read null. The break lines up with a mapping edit made three days ago.

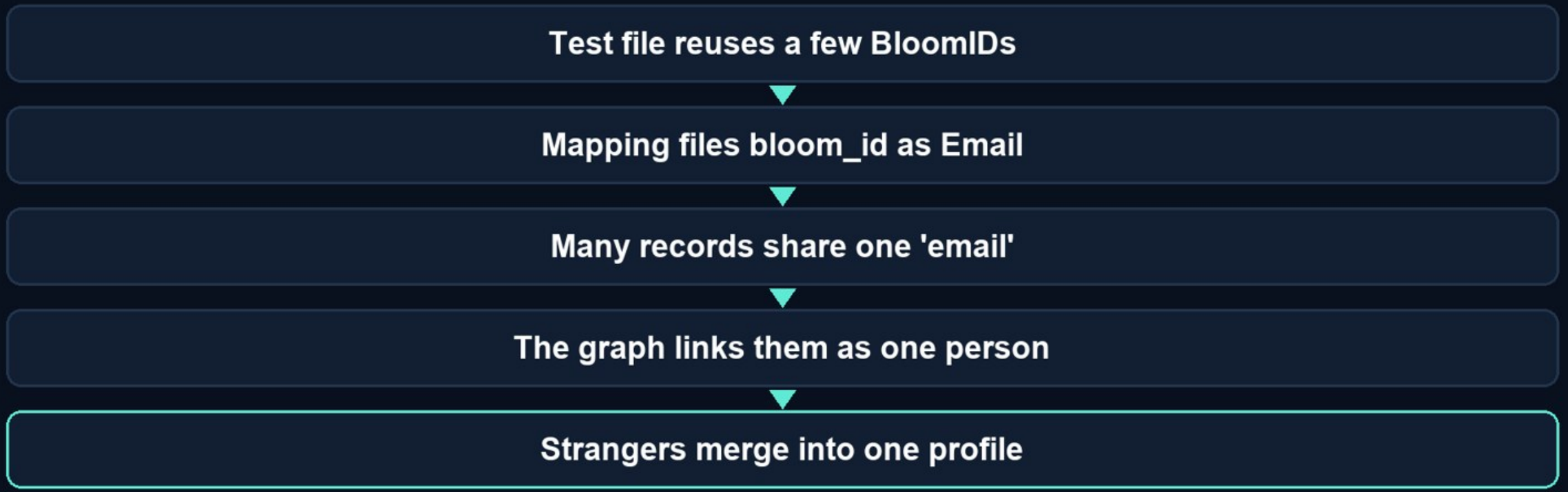
Fix the mapping, replay the window

Option	Verdict
Re-create all three schemas	Wrong layer, days of work
Re-ingest all 10,000 customers	A sledgehammer
Edit profiles by hand	Fights the pipeline
Restore field, replay three days	Right: surgical, at the layer

KEY IDEA

Restore the calculated field and re-run only the batches from the broken window. Rebuilds cost days for a one-line fix.

Landmine two: strangers in one graph



KEY IDEA

A test dataset filed BloomIDs as emails and reused a few of them, so every record sharing one value was joined into one graph.

Quarantine, do not reset

Option	Verdict
Reset the whole sandbox	Wrong: wipes everyone's work
Rename the Email namespace	Impossible: namespaces are fixed
Ignore it, it is test data	Wrong: it already merged people
Quarantine the polluting batches	Right: surgical and documented

KEY IDEA

Take the polluting batches out, keep the evidence, and fix the mapping. A sandbox reset destroys everyone's work to fix one dataset.

Landmine three: the audience that grew

Step	Landmine three
Symptom	Lapsed 90: 3,890 to 7,240 overnight
Evidence	Rule reads more than 30 days
Evidence	Audit log: who changed it, and when
Layer	The audience rule, not the data
Fix	Restore 90, verify about 3,890

KEY IDEA

Lapsed 90 was quietly edited to 30 days, and its estimate jumped from 3,890 to 7,240. The audit log named who changed it and when.

The five-line RCA a client accepts

Line	Bloom's vanishing loyalty band
Symptom	Loyalist file thinner every day
Cause	Mapping edit broke the band field
Fix	Restore field, replay three days
Verification	New profiles read band Gold again
Prevention	Null-rate check and dataflow review

KEY IDEA

An RCA has five lines: symptom, cause, fix, verification, prevention. Prevention is a detection net plus a process gate.

Done means someone else can run it

- Three RCAs, with API responses as evidence
- Named datasets, each with an owner
- A documented identity design
- Merge policies with written business reasons
- A known-bad list you chose to live with

KEY IDEA

A foundation is finished when someone else can run it: owners, a written identity design, merge-policy reasons, and a known-bad list.

Interview corner

- “An RCA has five lines: symptom, cause, fix,
- verification, prevention.
- Prevention is a detection net plus a process gate.”
- Then tell one landmine: symptom, layer, evidence, fix
- Asked about APIs? Four headers, sandbox first

KEY IDEA

An RCA has five lines: symptom, cause, fix, verification, prevention. Prevention is a detection net plus a process gate.

Recap, your question, and the next mystery

- Four headers on every call; check the sandbox
- Fix where the fault lives, then prove it
- Replay a window, quarantine batches, never reset
- Five-line RCA: prevention is net plus gate
- Next: Query Service, SQL on the data lake

KEY IDEA

Four headers, one person by two namespaces, fix at the faulty layer, surgery over rebuilds, five-line RCAs. Next: SQL over the data lake.